
Some of this is from the pyuv documentation itself, some of it is added by me. For extra context while possibly looking over the code. Useful to have the (quite large number of) variables explained.

1 Main Data Stuff

1.1 Constants

- Nbls: Number of baselines.
- Ntimes: Number of times.
- Nblts: Number of baseline-times. Not necessarily equal to Nbls * Ntimes.
- Nfreqs: Number of frequency channels.
- Npols: Number of polarizations.
- Nspws: Number of spectral windows.
- Nphase: Number of phase centers in the data.
- Nants_data: Number of antennas with data present (i.e. number of unique entries in ant_1_array and ant_2_array). May be smaller than the number of antennas in the array.
- vis_units: Visibility units, options are: “uncalib”, “Jy” or “K str”.

1.2 data_array

Array of the visibility data, shape: (Nblts, Nfreqs, Npols), type = complex float, in units of self.vis_units.

1.3 flag_array

Boolean flag, True is flagged, same shape as data_array. Can think of it as an accompanying array for each individual complex visibility measurement.

1.4 time_array, lst_array

These are both arrays of shape (Nblts), where each entry gives the time of the corresponding visibility measurement. They give the time at the center of integration. Extremely useful since this information is not encoded into the data_array itself. time_array is in units of Julian Date, while lst_array is in of local apparent sidereal time (LAST), units of radians.

1.5 ant_1_array, ant_2_array

Array of numbers for the first (or second) antenna present in a visibility. The number present is matched to that in the antenna_numbers attribute, so lets us know which antenna it is. Gives the antenna for each baseline-time, so it shape (Nblts), and type int. These just serve to know which baseline we are looking at.

1.6 baseline_array

Array of baseline numbers, shape (Nblts), type = int. These are special numbers that uniquely encode which two antenna are present. By default $\text{baseline} = 2048 * \text{ant1} + \text{ant2} + 2^{16}$, though other conventions are available. This is for compatibility with other data formats, and for efficiency in sorting methods.

1.7 freq_array, channel_width

freq_array is an array of frequencies at the center of the channel, with shape (Nfreqs,), units Hz. channel_width is also an array of shape (Nfreqs), with type = float, and gives the width of the frequency channels in (Hz).

1.8 polarization_array

Array of polarization integers, shape (Npols). There are specific integers that represent different types of polarizations. AIPS Memo 117 says: pseudo-stokes 1:4 (pI, pQ, pU, pV); circular -1:-4 (RR, LL, RL, LR); linear -5:-8 (XX, YY, XY, YX). NOTE: AIPS Memo 117 actually calls the pseudo-Stokes polarizations “Stokes”, but this is inaccurate as visibilities cannot be in true Stokes polarizations for physical antennas. We adopt the term pseudo-Stokes to refer to linear combinations of instrumental visibility polarizations (e.g. $pI = xx + yy$). A value of 0 indicates that the polarization is different for different spectral windows and is only allowed if flex_spw_polarization_array is defined (not None).

1.9 spw_array, flex_spw_id_array

spw_array is an array of spectral window numbers, shape (Nspws), which gives each spectral window some index. flex_spw_id_array maps the individual channels along the frequency axis to individual spectral windows, using the indexes from spw_array. It has shape (Nfreqs), type int. These two can tell us what frequency channels go into what spectral window.

1.10 integration_time

Length of the integration in seconds, shape (Nblts). For us, this remains constant since our accumulation length is constant. Best practice is for the integration_time to reflect the length of time a visibility was integrated over (so it should vary in the case of baseline-dependent averaging

and be a way to do selections for differently integrated baselines). Note that many files do not follow this convention, but it is safe to assume that the product of the `integration_time` and the `nsample_array` is the total amount of time included in a visibility.

1.11 `nsample_array`

Number of data points averaged into each data element. NOT required to be an integer, `type = float`, same shape as `data_array`. Best practice is for the `nsample_array` to be used to track flagging within an `integration_time` (leading to a decrease of the `nsample_array` value below 1) and LST averaging (leading to an increase in the `nsample_array` value). So datasets that have not been LST averaged should have `nsample_array` values less than or equal to 1. Note that many files do not follow this convention, but it is safe to assume that the product of the `integration_time` and the `nsample_array` is the total amount of time included in a visibility.

2 Phase Center Stuff

2.1 `phase_center_app_dec`

Apparent Declination of phase center in the topocentric frame of the observatory, units radians. Shape (Nblts,), `type = float`.

2.2 `phase_center_app_ra`

Apparent right ascension of phase center in the topocentric frame of the observatory, units radians. Shape (Nblts,), `type = float`.

2.3 `phase_center_catalog`

Dictionary that acts as a catalog, containing information on individual phase centers. Keys are the catalog IDs of the different phase centers in the object (matched to the parameter `phase_center_id_array`). At a minimum, each dictionary must contain the keys: `'cat_name'` giving the phase center name (this does not have to be unique, non-unique values can be used to indicate sets of phase centers that make up a mosaic observation), `'cat_type'`, which can be `'sidereal'` (fixed position in RA/Dec), `'ephem'` (position in RA/Dec which moves with time), `'driftscan'` (fixed position in Az/El, NOT the same as the old `phase_type = 'drift'`) or `'unprojected'` (baseline coordinates in ENU, but data are not phased, similar to the old `phase_type = 'drift'`) `'cat_lon'` (longitude coord, e.g. RA, either a single value or a one dimensional array of length Npts –the number of ephemeris data points– for `ephem` type phase centers), `'cat_lat'` (latitude coord, e.g. Dec., either a single value or a one dimensional array of length Npts –the number of ephemeris data points– for `ephem` type phase centers), `'cat_frame'` (coordinate frame, e.g. `icrs`, must be a frame supported by `astropy`). Other optional keys include `'cat_epoch'` (epoch and equinox of the coordinate frame, not needed for frames without an epoch (e.g. `ICRS`) unless there is

proper motion), 'cat_times' (times for the coordinates, only used for 'ephem' types), 'cat_pm_ra' (proper motion in RA), 'cat_pm_dec' (proper motion in Dec), 'cat_dist' (physical distance to the source in parsec, useful if parallax is important, either a single value or a one dimensional array of length Npts –the number of ephemeris data points– for ephem type phase centers.), 'cat_vrad' (rest frame velocity in km/s, either a single value or a one dimensional array of length Npts –the number of ephemeris data points– for ephem type phase centers.), and 'info_source' (describes where catalog info came from). See the documentation of the phase method for more details.

2.4 phase_center_frame_pa

Position angle between the hour circle (which is a great circle that goes through the target position and both poles) in the apparent/topocentric frame, and the frame given in the phase_center_frame attribute. Shape (Nblts,), type = float.

2.5 phase_center_id_array

Maps individual indices along the Nblt axis to a key in phase_center_catalog, which maps to a dict containing the other metadata for each phase center. Shape (Nblts), type = int.

2.6 uvw_array

Projected baseline vectors relative to phase center, shape (Nblts, 3), units meters. Convention is: $uvw = xyz(ant2) - xyz(ant1)$. Note that this is the Miriad convention but it is different from the AIPS/FITS convention (where $uvw = xyz(ant1) - xyz(ant2)$).

3 Other

3.1 telescope

pyuvdata.Telescope object containing the telescope metadata. This will contain stuff about the telescope, such as name, location, etc.

3.2 history

String of history, units English.